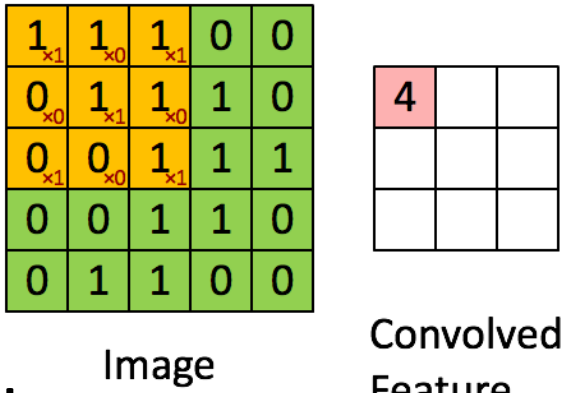
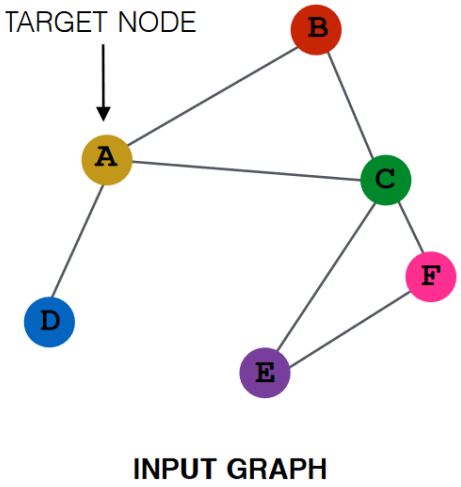
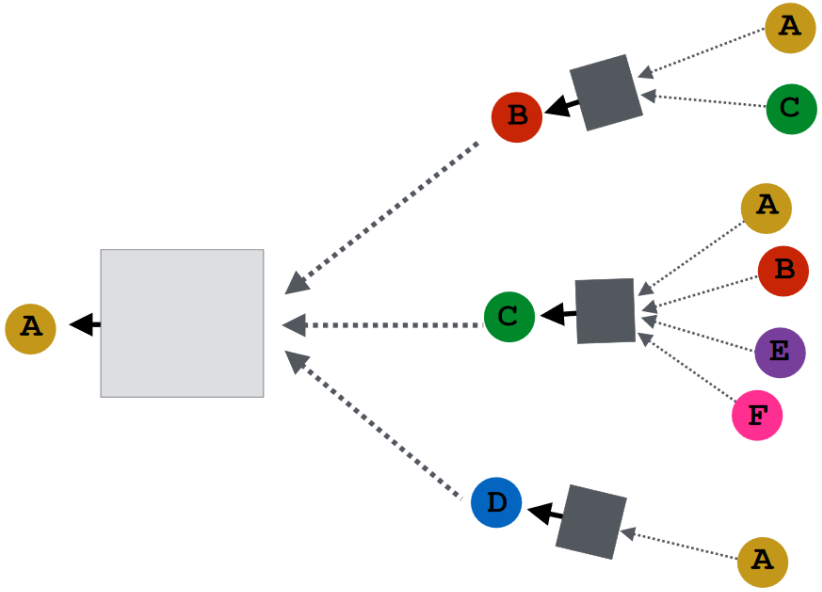
GRAPH NEURAL NETWORKS

Comparison with CNN

- Recall CNN
  - Regular graph
- GNN
  - Extend to irregular graph structure

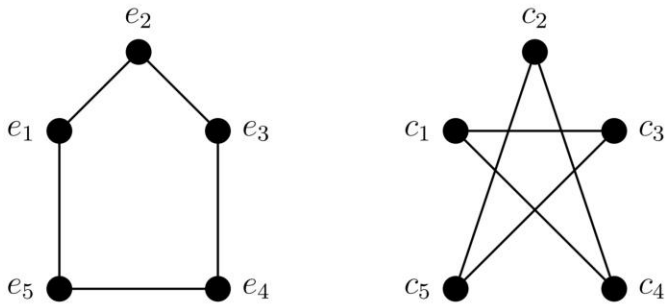


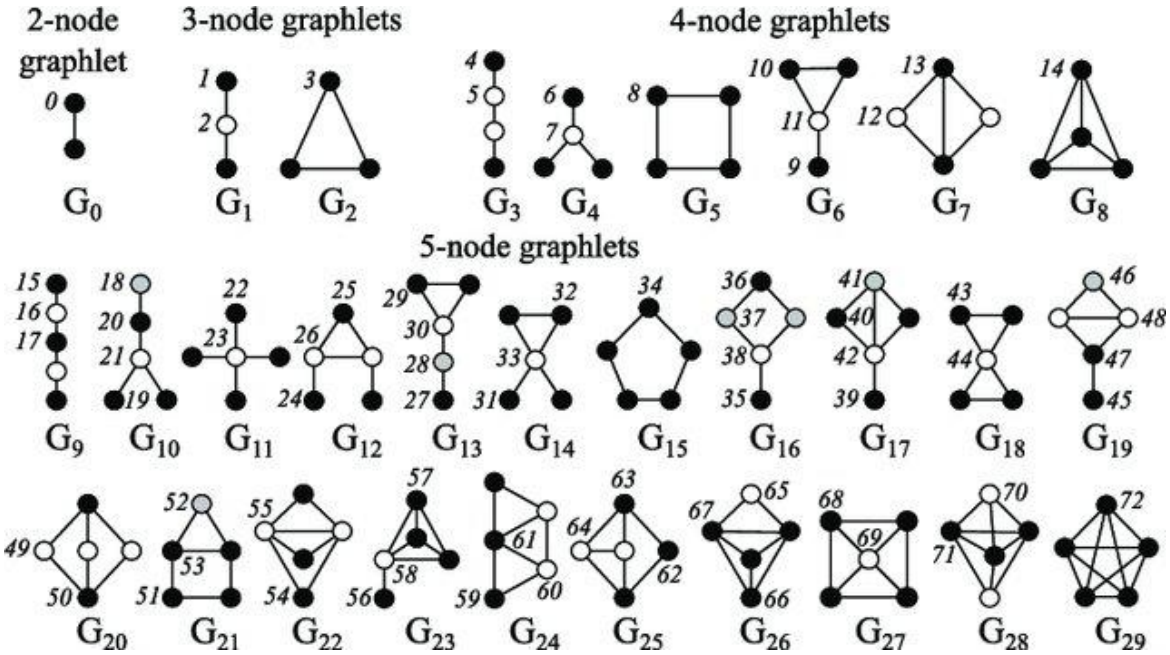




Graph isomorphism / Permutation invariance

- NP-hard





Notations

- An attributed graph $G = (V, E)$
  - V: vertex set
  - E: edge set
  - A: adjacency matrix
  - $X \in R^{d_0 \times |V|}$: feature matrix for all the nodes
  - $N(v)$: neighbors of node v
  - h_v^l: Representation vector of node v at Layer l
    - $Note\, h_v^0 = x_v$
  - $H^l \in R^{d_l \times |V|}$: representation matrix

The General Architecture of GNNs

- For a node v at layer t

$$h_v^{(t)} = f\left(h_v^{(t-1)}, \left\{h_u^{(t-1)} | u \in \mathcal{N}(v)\right\}\right)$$

representation vector from previous layer for node v

representation vectors from previous layer for node v's neighbors

  - A function of representations of neighbors and itself from previous layers
    - **Aggregation** of neighbors
    - **Transformation** to a different space
    - **Combination** of neighbors and the node itself

Graph Convolutional Network (GCN)

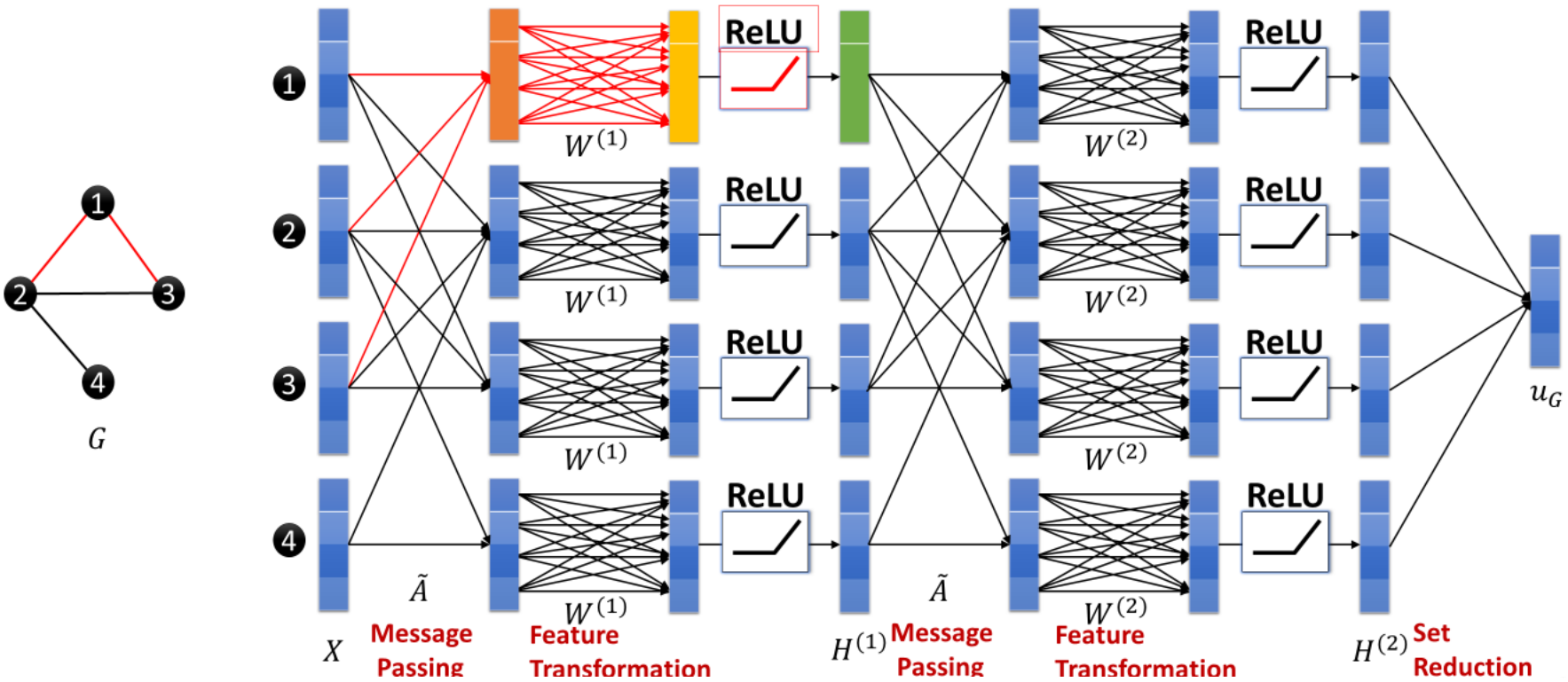
- Kipf and Welling, ICLR'17
  - $f(H^{(l)}, A) = \sigma\left(\hat{D}^{-\frac{1}{2}}\hat{A}\hat{D}^{-\frac{1}{2}}H^{(l)}W^{(l)}\right)$, $\hat{A} = A + I$
  - f: graph filter
- From a node v's perspective

$$\mathbf{h}_v^k = \sigma\left(\mathbf{W}_k \sum_{u \in N(v) \cup v} \frac{\mathbf{h}_u^{k-1}}{\sqrt{|N(u)||N(v)|}}\right)$$

W_k: *weight matrix at Layer k, shared across different nodes*

Toy example of 2-layer GCN on a 4-node graph

- Computation graph



GraphSAGE

- Inductive Representation Learning on Large Graphs
  William L. Hamilton*, Rex Ying*, Jure Leskovec, NeurIPS'17

$$\mathbf{h}_{\mathcal{N}(v)}^{k} \leftarrow \text{AGGREGATE}_k\left(\{\mathbf{h}_u^{k-1}, \forall u \in \mathcal{N}(v)\}\right)$$

$$\mathbf{h}_v^{k} \leftarrow \sigma\left(\mathbf{W}^k \cdot \text{CONCAT}\left(\mathbf{h}_v^{k-1}, \mathbf{h}_{\mathcal{N}(v)}^{k}\right)\right)$$

A more general form

$$\mathbf{h}_v^k = \sigma\left(\left[\mathbf{W}_k \cdot \text{AGG}\left(\{\mathbf{h}_u^{k-1}, \forall u \in N(v)\}\right), \mathbf{B}_k \mathbf{h}_v^{k-1}\right]\right)$$

More about AGG

- Mean $$\text{AGG} = \sum_{u \in N(v)} \frac{\mathbf{h}_u^{k-1}}{|N(v)|}$$

- LSTM $$\text{AGG} = \text{LSTM}\left([\mathbf{h}_u^{k-1}, \forall u \in \pi(N(v))]\right)$$
  - $\pi(\cdot)$: *a random permutation*

- Pool $$\text{AGG} = \gamma\left\{\mathbf{Q}\mathbf{h}_u^{k-1}, \forall u \in N(v)\right\}$$
  - $\gamma(\cdot)$: Element-wise mean/max pooling of neighbor set

Message-Passing Neural Network

- Gilmer et al., 2017. Neural Message Passing for Quantum Chemistry. *ICML.*
- *A general framework that subsumes most GNNs*
  - Can also include edge information
- Two steps
  - Get messages from neighbors at step k

$$\mathbf{m}_v^k = \sum_{u \in N(v)} M(\mathbf{h}_u^{k-1}, \mathbf{h}_v^{k-1}, \mathbf{e}_{u,v})$$

e.g., Sum or MLP

  - Update the node latent represent based on the msg

$$\mathbf{h}_v^k = U(\mathbf{h}_v^{k-1}, \mathbf{m}_v^k)$$

e.g., LSTM, GRU

***A special case*: *GGNN*, Li et al., Gated graph sequence neural networks, ICLR 2015**

Graph Attention Network (GAN)

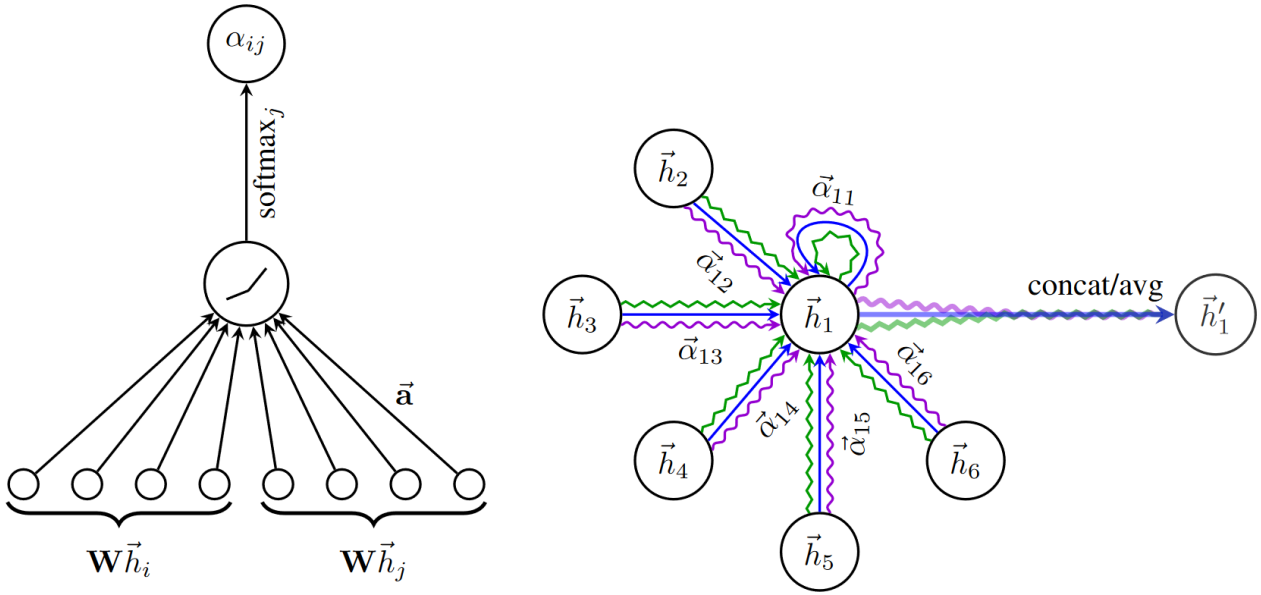
- How to decide the importance of neighbors?
  - GCN: a predefined weight
  - Others: no differentiation
- GAN: decide the weights using learnable attention
  - Velickovic et al., 2018. Graph Attention Networks. *ICLR.*

$$\vec{h}'_i = \sigma\left(\sum_{j \in \mathcal{N}_i} \alpha_{ij} \mathbf{W} \vec{h}_j\right)$$

The attention mechanism

- Potentially many possible designs

$$\alpha_{ij} = \frac{\exp\left(\text{LeakyReLU}\left(\vec{\mathbf{a}}^T[\mathbf{W}\vec{h}_i \| \mathbf{W}\vec{h}_j]\right)\right)}{\sum_{k\in\mathcal{N}_i}\exp\left(\text{LeakyReLU}\left(\vec{\mathbf{a}}^T[\mathbf{W}\vec{h}_i \| \mathbf{W}\vec{h}_k]\right)\right)}$$



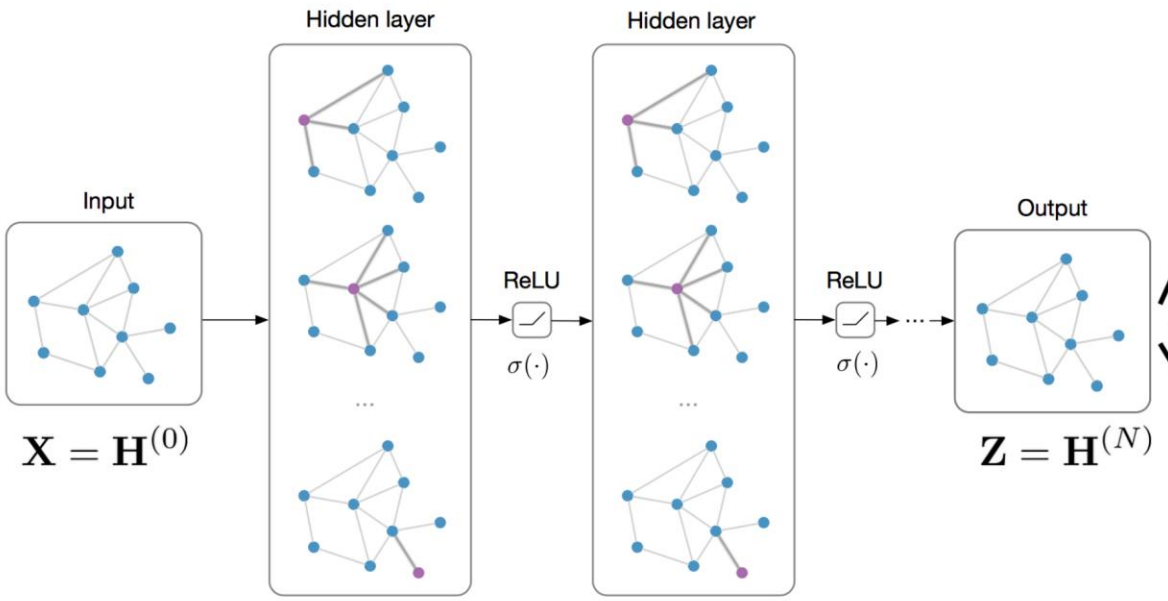
See more

- https://pytorch-geometric.readthedocs.io/en/latest/modules/nn.html



How to use output features?

Input: Feature matrix $\mathbf{X} \in \mathbb{R}^{N \times E}$, preprocessed adjacency matrix $\hat{\mathbf{A}}$



$$\mathbf{H}^{(l+1)} = \sigma\left(\hat{\mathbf{A}}\mathbf{H}^{(l)}\mathbf{W}^{(l)}\right)$$

Node classification:

$\mathrm{softmax}(\mathbf{z_n})$

e.g. Kipf & Welling (ICLR 2017)

Graph classification:

$\mathrm{softmax}(\sum_n \mathbf{z_n})$

e.g. Duvenaud et al. (NIPS 2015)

Link prediction:

$p(A_{ij}) = \sigma(\mathbf{z_i^T z_j})$

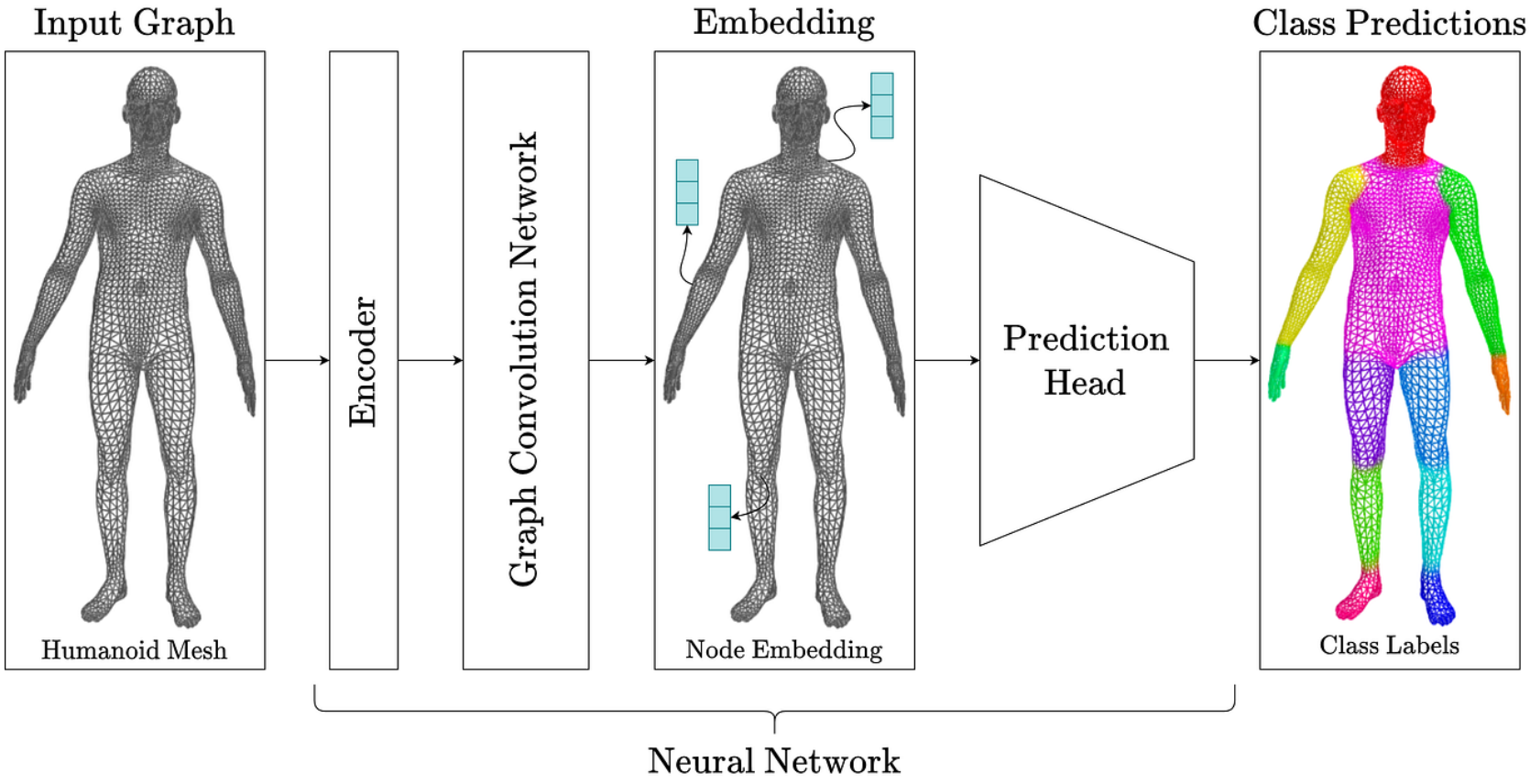
Kipf & Welling (NIPS BDL 2016)

"Graph Auto-Encoders"

CV APPLICATIONS

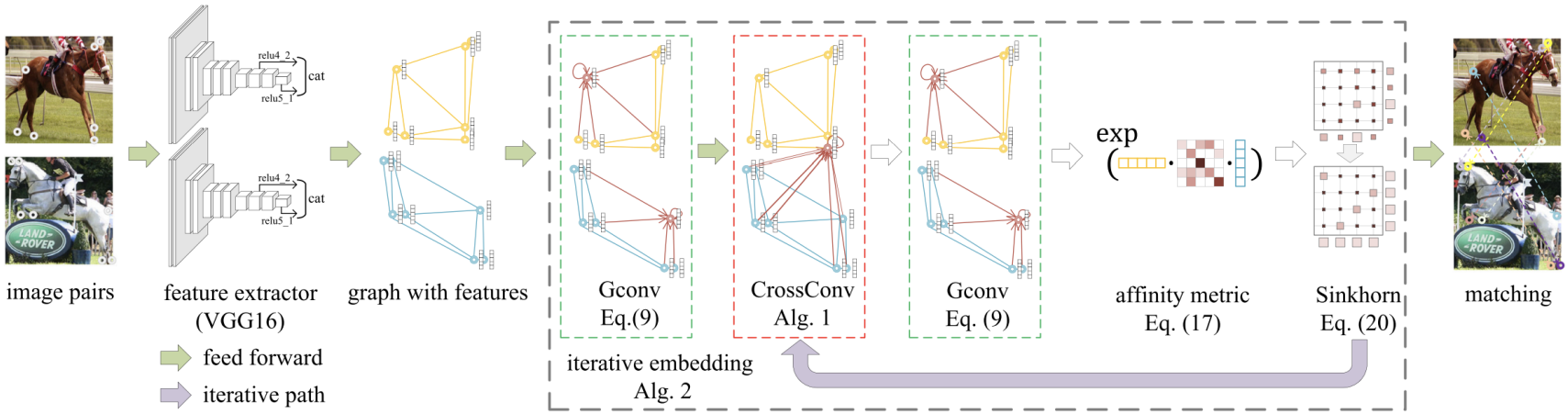
3D Pointcloud Segmentation

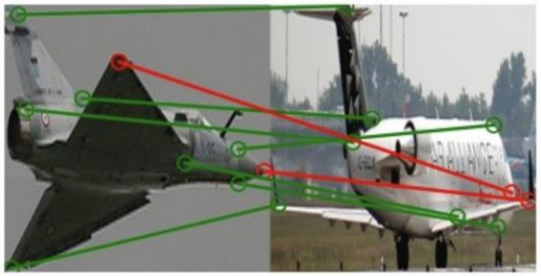
- Node classification

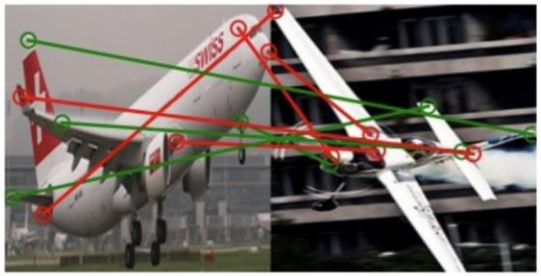


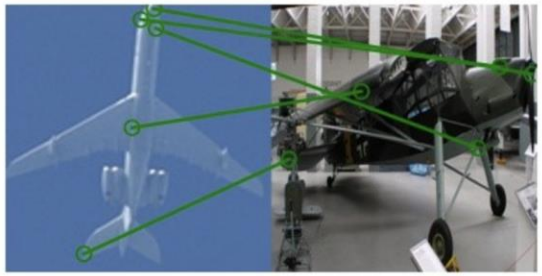
Part-based Image Matching

- Pairwise graph alignment



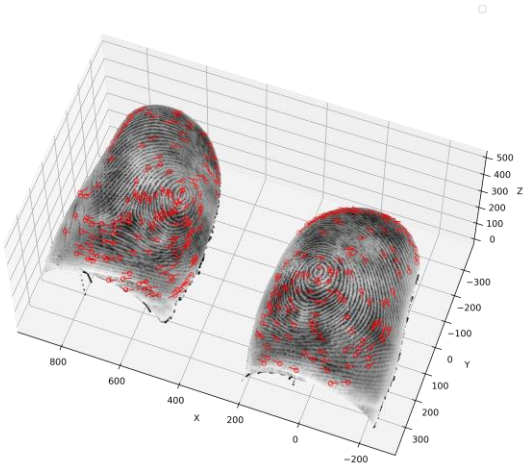


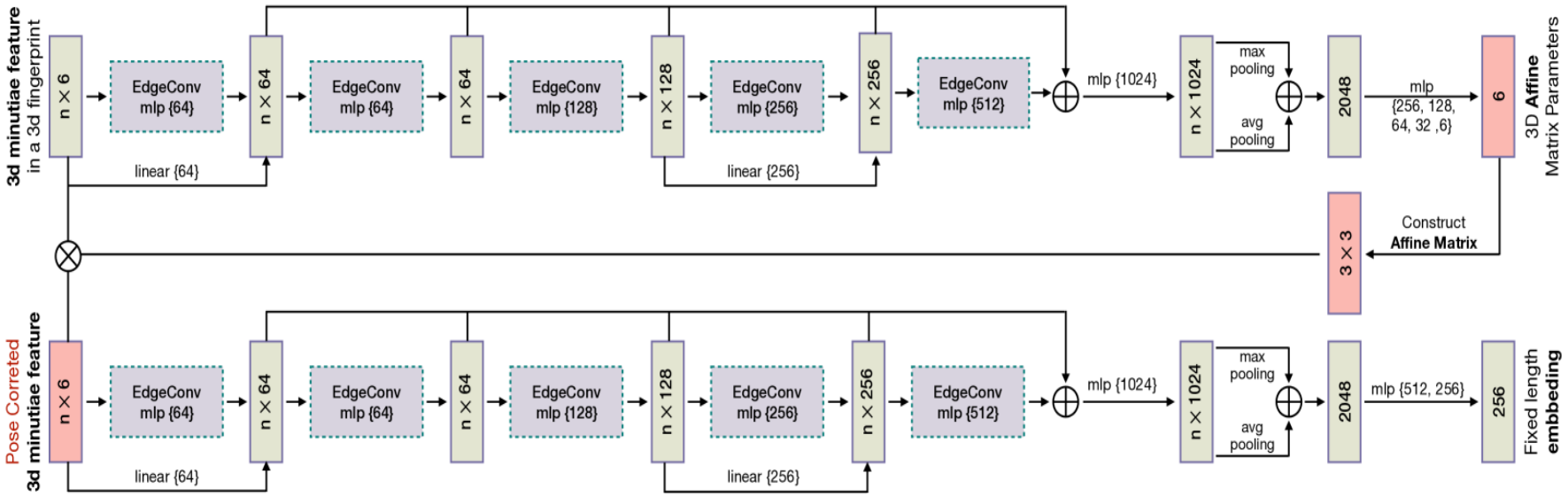




Fingerprint Description for Matching

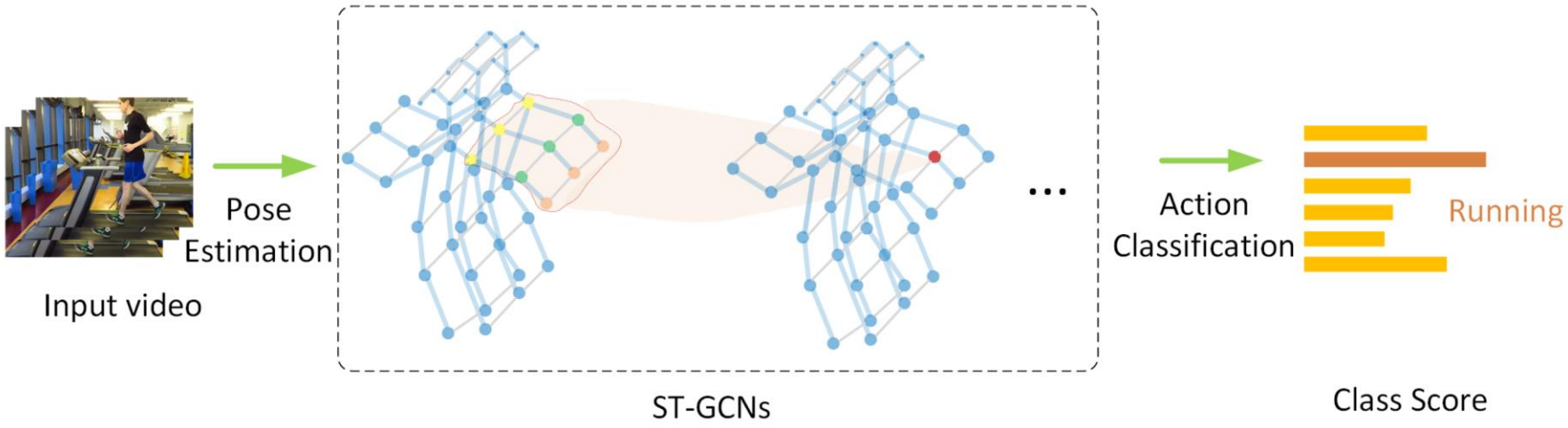
- Graph template extraction





Action Recognition

- Spatio-temporal graph classification



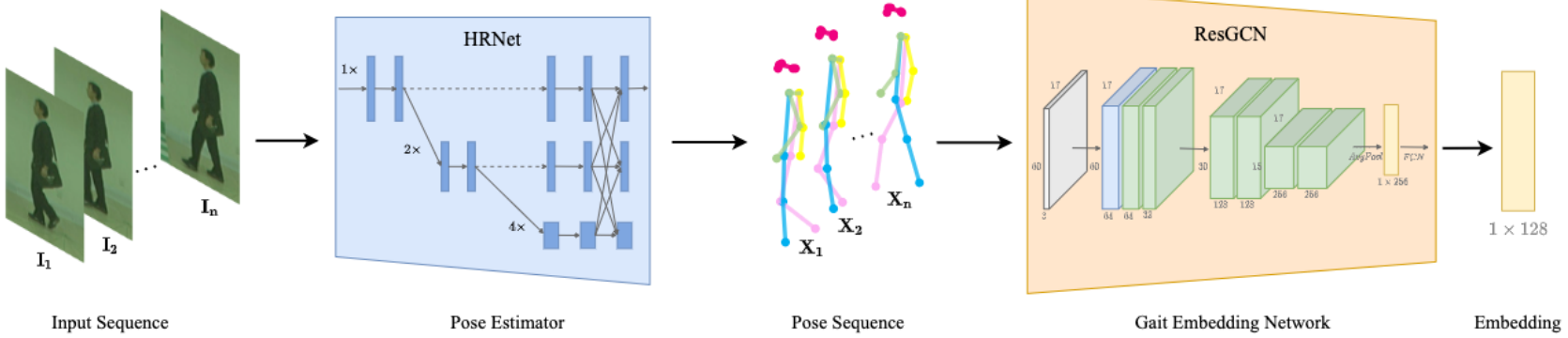


Image Understanding

- Edge prediction / semantic graph representation

